

Edge cases (a template with the wrong variables)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Answer: {question}")

# missing variable
try:
    prompt.invoke({})
except Exception as e:
    print("Missing variable error:", type(e).__name__, "-", e)

# extra variable
result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
print("Extra variable result:", result)
```

Expected output:

```
Missing variable error: KeyError - 'question'
Extra variable result: messages=[HumanMessage(content='Answer: What is LCEL?')]
```

This version works and shows both cases clearly. It catches the broad `Exception` for the missing-variable case rather than the specific `KeyError`, which is fine for exploring but not what you'd write once you know exactly what error type to expect.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the specific `KeyError`, in named functions

```
from langchain_core.prompts import ChatPromptTemplate

def demonstrate_missing_variable(prompt: ChatPromptTemplate) -> None:
    try:
        prompt.invoke({})
    except KeyError as e:
        print(f"Missing variable -> KeyError: {e}")

def demonstrate_extra_variable(prompt: ChatPromptTemplate) -> None:
    result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
    print(f"Extra variable -> no error, result: {result}")

def main() -> None:
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    demonstrate_missing_variable(prompt)
    demonstrate_extra_variable(prompt)

if __name__ == "__main__":
    main()
```

Expected output:

```
Missing variable -> KeyError: 'question'
Extra variable -> no error, result: messages=[HumanMessage(content='Answer: What
```

Difference from Basic: catching `KeyError` specifically (not a broad `Exception`) documents, right in the code, exactly what failure you expect and are prepared to handle — if the library ever changed to raise a different error type here, this code would stop silently swallowing it and let you notice. Splitting the two demonstrations into named functions also makes each case something you could reuse as an actual test later, not just a script you run once and read.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — check `input_variables` before ever calling `.invoke()`

```
from langchain_core.prompts import ChatPromptTemplate

def validate_inputs(prompt: ChatPromptTemplate, provided: dict) -> None:
    missing = set(prompt.input_variables) - set(provided.keys())
    if missing:
        raise ValueError(f"Missing prompt variable(s): {sorted(missing)}")

def warn_about_extra_inputs(prompt: ChatPromptTemplate, provided: dict) -> None:
    extra = set(provided.keys()) - set(prompt.input_variables)
    if extra:
        print(f"Warning: {sorted(extra)} will be silently ignored by this template")

def main() -> None:
    prompt = ChatPromptTemplate.from_template("Answer: {question}")

    typo_inputs = {"questoin": "What is LCEL?"} # typo, not "question"
    try:
        validate_inputs(prompt, typo_inputs)
    except ValueError as e:
        print(f"Caught before calling invoke(): {e}")
    warn_about_extra_inputs(prompt, typo_inputs)

if __name__ == "__main__":
    main()
```

Expected output:

```
Caught before calling invoke(): Missing prompt variable(s): ['question']
Warning: ['questoin'] will be silently ignored by this template
```

Both problems get caught here — `validate_inputs` catches the required `question` genuinely being missing, and `warn_about_extra_inputs` catches that `questoin` (the typo) would otherwise vanish silently, with no error at all, exactly the gap Hint 1's Advanced question pointed at.

Approach 2 — build the check into a reusable `safe_invoke()` wrapper

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable

def safe_invoke(chain: Runnable, prompt: ChatPromptTemplate, inputs: dict):
    missing = set(prompt.input_variables) - set(inputs.keys())
    if missing:
        raise ValueError(f"Missing prompt variable(s): {sorted(missing)}")

    extra = set(inputs.keys()) - set(prompt.input_variables)
    if extra:
        print(f"Warning: {sorted(extra)} will be silently ignored by this template")

    return chain.invoke(inputs)
```

Expected output, used in place of `chain.invoke(inputs)` directly: identical results on correctly-shaped input, plus the same missing/extra checks from Approach 1 run automatically on every call, without every caller having to remember to run them separately.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate discovers both behaviors by calling `.invoke(...)` once and observing the result — which means the extra-variable case is never actually *checked*, only demonstrated. Approach 1 reads `prompt.input_variables` directly and checks both directions explicitly, catching a typo'd extra key that `.invoke()` alone would never mention. Approach 2 takes the exact same checks and wraps them around `.invoke()` itself, so every call through `safe_invoke()` gets the validation automatically, instead of relying on every caller to remember to check first.

Which one should you use, and why? For a quick, one-time exploration, Basic or Intermediate is fine. Approach 1's `validate_inputs` / `warn_about_extra_inputs` functions are worth having anywhere you're building prompts from user input or config, where a typo'd key is a real, easy mistake. Reach for Approach 2's `safe_invoke()` wrapper once you have several chains being called from several places — centralizing the check in one wrapper means you can't forget to call it, the same reasoning behind wrapping `require_env()` calls inside `load_config()` instead of trusting every caller to check environment variables correctly on their own.